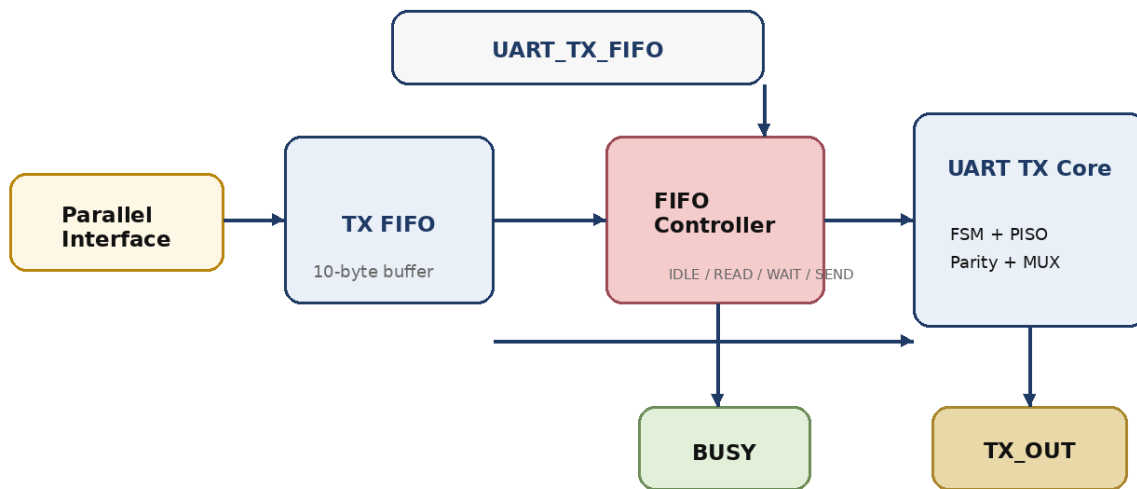


UART TRANSMITTER + TX FIFO

8-bit UART Transmitter with Optional Parity and FIFO Buffering

Verilog RTL Implementation and Verification

UART TX + FIFO Top-Level Architecture



P_DATA / DATA_VALID enter the FIFO. The controller starts the UART only when a byte is available and the transmitter is idle.

Figure 1. Top-level architecture of the UART TX with TX FIFO.

Prepared by: Omar Mohamed Mostafa

1. Introduction

1.1 Purpose

This document describes the design, architecture, and verification of an 8-bit UART transmitter implemented in Verilog RTL. It explains how the transmitter builds a serial frame, how the internal modules work together, and how a TX FIFO was added to buffer multiple bytes while a transmission is in progress.

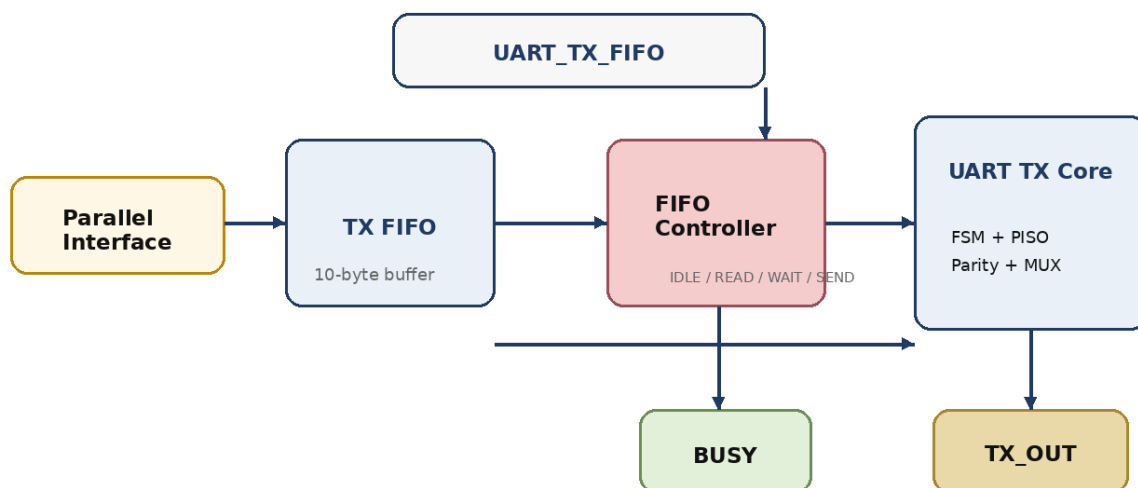
1.2 Scope

The design implements an 8-bit UART transmitter with optional parity support. The transmitter converts parallel input data into a serial frame consisting of one start bit, eight data bits transmitted LSB first, an optional even or odd parity bit, and one stop bit. An asynchronous active-low reset is used to initialize the design. The design is organized into an FSM, PISO shift register, parity generator, MUX, TX FIFO, and FIFO controller. Verification is performed with self-checking Verilog testbenches using a 200 MHz clock.

2. System Overview

The top-level module accepts P_DATA, DATA_VALID, PAR_EN, and PAR_TYPE from the surrounding logic. Incoming bytes are first stored in the TX FIFO. A small controller watches the FIFO status and the UART BUSY signal, then transfers one byte at a time into the original UART TX core. This keeps the transmitter architecture simple while allowing several bytes to wait in a queue.

UART TX + FIFO Top-Level Architecture



P_DATA / DATA_VALID enter the FIFO. The controller starts the UART only when a byte is available and the transmitter is idle.

Figure 1. The FIFO sits in front of the existing UART TX core; the controller handles the hand-off.

3. Module Architecture

3.1 Top-Level UART_TX_FIFO

The top-level wrapper connects the FIFO, FIFO controller, and the existing UART TX core. The external interface remains simple: the source sends an 8-bit P_DATA value together with DATA_VALID, while TX_OUT and BUSY remain the main outputs. The FIFO absorbs bursts of incoming bytes and prevents them from being lost while the UART is busy.

Signal	Width	Description
CLK	1	System clock used by all modules
RST_N	1	Asynchronous active-low reset
P_DATA	8	Parallel input byte
DATA_VALID	1	Request to store a new byte
PAR_EN	1	Enable or disable parity
PAR_TYPE	1	0 = even parity, 1 = odd parity
TX_OUT	1	Serial UART output
BUSY	1	High while the UART is transmitting

3.2 UART TX Core

The UART TX core is the original transmitter datapath. It is kept as an independent block so the FIFO enhancement does not change the way the serial frame itself is generated. The core contains four small functional blocks: the FSM decides which frame field is active, the PISO serializes the byte, the parity block calculates the parity bit, and the MUX selects the signal that drives TX_OUT.

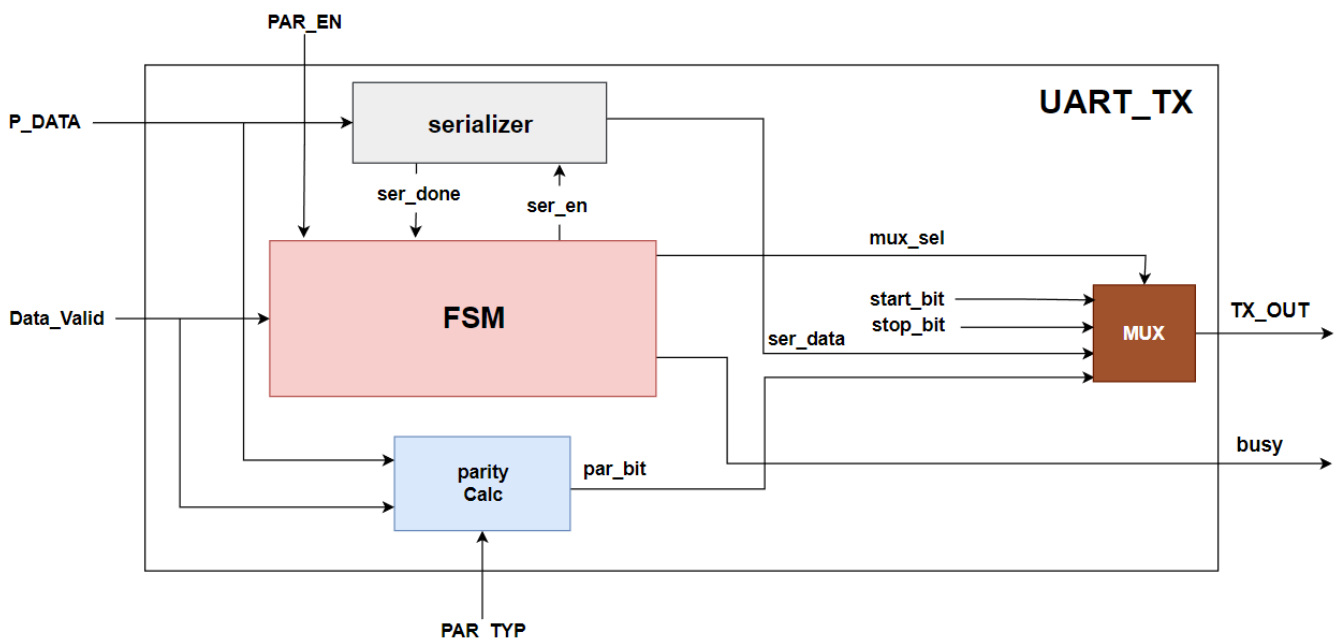


Figure 2. Control sequence used by the UART TX core.

3.3 FSM Operation

The transmitter FSM uses five states: IDLE, START, DATA, PARITY, and STOP. In IDLE the line is high. When DATA_VALID is accepted, the FSM enters START. It then moves to DATA and waits for the serializer to finish the eight data bits. If parity is enabled, the FSM visits PARITY before reaching STOP. Finally, the FSM returns to IDLE and BUSY goes low.

State	Behaviour
IDLE	TX_OUT is high. A valid request starts a new frame.
START	Selects the start bit (0) and marks the transmitter as busy.
DATA	Selects the serialized data bits, LSB first, until ser_done is asserted.
PARITY	Selects the calculated parity bit when parity is enabled.
STOP	Selects the stop bit (1), then returns to IDLE.

3.4 PISO Serializer

The PISO block stores the input byte in a shift register. It outputs the least-significant bit first and shifts the register after each transmitted bit. A small counter tracks the number of bits already sent and generates ser_done after the last data bit.

3.5 Parity Generator

The parity generator uses a reduction XOR over the eight data bits. For even parity, the XOR result is used directly. For odd parity, the result is inverted. PAR_EN determines whether this bit is included in the frame.

3.6 MUX

The MUX selects one of four sources for TX_OUT: start bit, serial data, parity bit, or stop bit. The select signal comes from the FSM, so only one frame field is driven at a time.

4. TX FIFO and Controller

4.1 TX FIFO

The TX FIFO is a small circular buffer placed in front of the UART TX core. In this implementation the FIFO stores 8-bit data and has a depth of 10 entries. The write pointer marks the next location for a new byte, the read pointer marks the oldest stored byte, and a count register tracks the number of occupied entries.

TX FIFO Operation

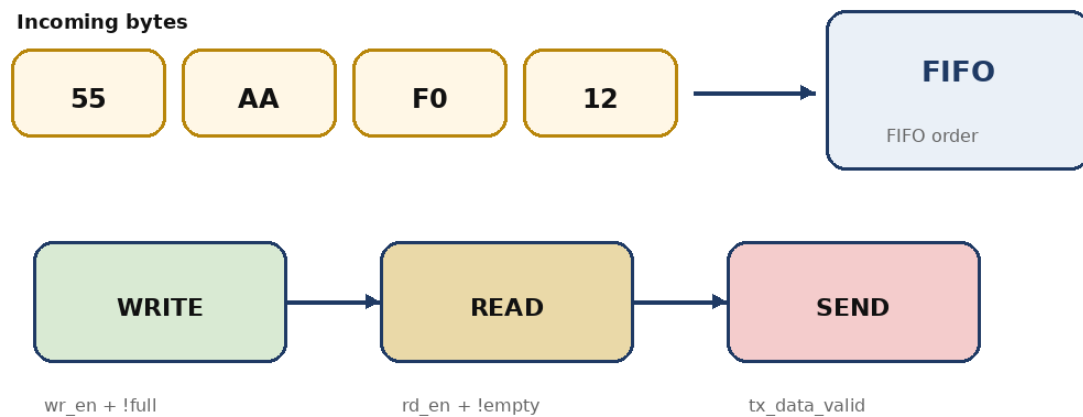


Figure 3. FIFO data flow: bytes are stored in order and later read in the same order.

Signal	Purpose
data_in	Byte written into the FIFO
wr_en	Requests a write when the FIFO is not full
rd_en	Requests a read when the FIFO is not empty
data_out	Byte currently supplied to the next stage
full	Indicates that the FIFO cannot accept another byte
empty	Indicates that there is no byte available to read

4.2 FIFO Controller

The FIFO controller is a small FSM with four states: IDLE, READ, WAIT, and SEND. In IDLE it waits until the FIFO is not empty and the UART is not busy. READ generates the FIFO read pulse. WAIT gives the registered FIFO output a full clock to settle. SEND raises tx_data_valid for one clock so the UART TX core loads the byte. The controller then returns to IDLE and waits for the UART to finish.

5. Frame Format

The transmitted frame starts with a low start bit, followed by eight data bits transmitted LSB first. When parity is enabled, one parity bit is added after the data. The frame ends with a high stop bit. When parity is disabled, the parity stage is simply skipped.



Figure 4. UART frame structure used by the transmitter.

Configuration	Frame
Parity disabled	START (0) + D0..D7 + STOP (1)
Even parity	START (0) + D0..D7 + EVEN PARITY + STOP (1)
Odd parity	START (0) + D0..D7 + ODD PARITY + STOP (1)

6. Interface Timing

DATA_VALID is intended as a short request to store a new byte. Once a byte is accepted into the FIFO, the source can continue presenting later bytes without waiting for the current UART frame to finish. BUSY remains high while the UART TX core is sending the current frame. The FIFO controller only launches another byte after the UART is idle and data is available.

The testbench clock is 200 MHz, giving a 5 ns clock period. The verification code samples the transmitted bits away from the active clock edge to avoid race conditions between the DUT and the checker.

7. Verification

Test	Type	Main coverage
FIFO TB	Self-checking	Write/read order, full, empty
Controller TB	Self-checking	Reset, READ/SEND sequencing, busy/empty
Integration TB	Self-checking	UART frame correctness + FIFO ordering + parity modes

7.1 Internal UART Module Verification

The UART TX core is built from four internal modules: the FSM, PISO serializer, parity generator, and MUX. Their behaviour is checked through the UART TX verification flow. The FSM is checked for the correct IDLE, START, DATA, PARITY, and STOP sequence. The PISO is checked for correct LSB-first shifting and completion after all eight data bits. The parity generator is checked for both even and odd parity, while the MUX is checked to ensure that the correct frame field is selected for TX_OUT in each state.

7.2 UART TX Core Verification

The UART TX core is verified as a standalone transmitter before adding the FIFO wrapper. The testbench applies different 8-bit data values and checks the complete serial frame at TX_OUT. The verification covers the start bit, all eight data bits, the optional parity bit, and the stop bit. No-parity, even-parity, and odd-parity configurations are included so that the core behaviour can be checked independently of the FIFO.

7.3 Complete UART TX + FIFO Verification

The final testbench instantiates the complete top-level UART TX + FIFO module. It sends multiple bytes through P_DATA and DATA_VALID, including cases where several bytes are queued while the UART is busy. The checker verifies that the bytes are transmitted in FIFO order and that every generated frame has the correct start bit, data bits, optional parity bit, and stop bit. The FIFO full/empty behaviour and the controller hand-off are also exercised as part of the complete data path.

7.4 Verification Results

The FIFO-only testbench passed the write, read, full, empty, and ordering checks. The FIFO controller testbench passed the reset, IDLE, READ, WAIT, SEND, and busy/empty control checks. The original UART TX core was verified separately before the FIFO was added. The complete UART TX + FIFO integration test is used to validate the final data path and is being checked for correct timing between the FIFO controller and the UART TX core.

8. Design Considerations and Limitations

The FIFO enhancement improves the original single-byte-at-a-time interface by allowing several bytes to wait while the transmitter is busy. The current design is intentionally small and keeps the original UART TX core unchanged.

- The FIFO stores 8-bit data and uses a fixed depth of 10 entries in the current project.
- The FIFO must reject writes when full and reads when empty.
- The FIFO controller adds a small amount of latency between reading a byte and asserting tx_data_valid so the registered FIFO output is stable.
- PAR_EN and PAR_TYPE are top-level configuration signals. They should remain consistent for the queued data until those bytes have been transmitted.
- The current verification is simulation-based; hardware timing, baud-rate generation, and physical serial-line effects are outside this project scope.

End of document